

Angular Forms

Two ways to build the same sign-up form: **Template-Driven** keeps logic in the HTML, while **Reactive** defines the form and validation in the TypeScript class. This guide covers both end to end, plus the control states and custom validators you'll need in a real app.

1 · TWO APPROACHES

Same form, two different philosophies

TEMPLATE-DRIVEN	REACTIVE
Most logic lives in the HTML, using <code>ngModel</code>	Form structure & validation defined in the TS class
Template does more of the work	Template just displays what the class defines
Great for small, simple forms	Great for complex, dynamic, or heavily-tested forms

2 · TEMPLATE-DRIVEN FORMS

Setup, template, and validation

Component

```
import { FormsModule } from '@angular/forms';

@Component({ selector: 'app-signup', imports: [FormsModule] })
export class SignupComponent {
  name = '';
  email = '';
  onSubmit() { console.log(this.name, this.email); }
}
```

Template with `ngModel`

```
<form #signupForm="ngForm" (ngSubmit)="onSubmit()">
  <input name="name" [(ngModel)]="name" required minlength="3" #nameField="ngModel">
  <input name="email" [(ngModel)]="email" required email #emailField="ngModel">
  <button type="submit" [disabled]="signupForm.invalid">Sign Up</button>
</form>

@if (nameField.invalid && nameField.touched) {
  <small>
    @if (nameField.errors?.['required']) { Name is required. }
    @if (nameField.errors?.['minlength']) { Name must be at least 3 characters. }
  </small>
}
```

`[(ngModel)]` is **two-way binding**: it both displays the value and updates it as the user types.

`#nameField="ngModel"` creates a local template reference we can use to check validation state. `touched` means the user clicked in and left the field — always check `invalid && touched` together, so errors don't show before the user has even typed anything.

Setup, template, and validation

Component

```
import { FormGroup, FormControl, Validators } from '@angular/forms';

signupForm = new FormGroup({
  name: new FormControl('', [Validators.required, Validators.minLength(3)]),
  email: new FormControl('', [Validators.required, Validators.email]),
});
onSubmit() { console.log(this.signupForm.value); }
```

Template with formGroup

```
<form [formGroup]="signupForm" (ngSubmit)="onSubmit()">
  <input formControlName="name">
  <input formControlName="email">
  <button type="submit" [disabled]="signupForm.invalid">Sign Up</button>
</form>
```

`FormControl` represents ONE input field — its value and its validation rules. `FormGroup` bundles multiple `FormControls` together into one form object. Because the whole form is a plain object in the class, it's very easy to unit test — no HTML needed to check validation logic. Reach a control with `signupForm.get('name')` instead of a template reference variable.

4 · FORM & CONTROL STATES

A quick cheat sheet

STATE	TRUE WHEN...
<code>pristine</code>	the user hasn't changed the value yet
<code>dirty</code>	the value has been changed at least once (opposite of <code>pristine</code>)
<code>untouched</code>	the field has never been focused
<code>touched</code>	the field was focused and then lost focus (opposite of <code>untouched</code>)
<code>valid / invalid</code>	every validator passes / at least one fails
<code>pending</code>	an async validator (e.g. checking a username against the server) is still running

The classic pattern — `control.invalid` && `control.touched` — exists so you never scare the user with a red error before they've had a chance to type.

5 · CUSTOM VALIDATORS

When built-in validators aren't enough

```
function passwordsMatch(group: AbstractControl): ValidationErrors | null {
  const pass = group.get('password')?.value;
  const confirm = group.get('confirmPassword')?.value;
  return pass === confirm ? null : { mismatch: true };
}

signupForm = new FormGroup({
  password: new FormControl(''),
  confirmPassword: new FormControl(''),
}, { validators: passwordsMatch });
```

A validator is just a function: it takes a control (or group) and returns `null` when valid, or an error object when invalid. Attach it at the `FormControl` level for single-field rules, or at the `FormGroup` level (as above) when a rule needs to compare two fields.

6 · DECISION GUIDE

When to use which?

CHOOSE TEMPLATE-DRIVEN WHEN...	CHOOSE REACTIVE WHEN...
The form is simple (few fields)	The form is complex or has many fields
You want the fastest way to get something working	Validation rules change dynamically
Validation rules are straightforward	You need to unit test form logic easily
	You're building a larger, production app

7 · BEST PRACTICES

- Always disable the submit button while the form is invalid: `[disabled]="form.invalid"`.
- Only show error messages after the field has been touched.
- Keep error messages specific ("Name must be at least 3 characters") rather than generic ("Invalid input").
- Don't mix Template-Driven and Reactive approaches in the same form — pick one and stay consistent.